

# Activity Tracker — Product Design Document

## 1. Vision in One Sentence

A flexible, AI-assisted Android app for tracking *any* recurring or one-off activity in your life, with rich per-activity structure, motivating streaks, and personalized insights — without forcing every activity into the same rigid mold.

---

## 2. Core Design Principle

Every other tracker app fails one of two ways: either it's so generic that gym tracking and reading tracking feel equally bland, or it's so specialized that adding "call mom every Sunday" feels weird. The way out is a **flexible activity model** where the *shell* is uniform (schedule, completion, streaks, analytics) but the *contents* are activity-specific (workout sets, pages read, journal text, distance run, etc.).

Think of it like this: every activity is a row in a database, but the activity *type* decides what columns it has.

---

## 3. Data Model

### 3.1 Activity

Activity

|                  |                                          |
|------------------|------------------------------------------|
| — id             |                                          |
| — name           | e.g., "Gym", "Read", "Call Mom"          |
| — type           | RECURRING   OCCASIONAL   ONE_OFF         |
| — schedule       | (only for RECURRING)                     |
| — frequency      | daily, weekly, custom                    |
| — days_of_week   | [Mon, Wed, Fri] etc.                     |
| — time_window    | optional reminder time                   |
| — subtask_schema | user-defined variable tree (see §3.3)    |
| — goal           | optional (e.g., "5x/week", "30 min/day") |
| — created_at     |                                          |
| — archived       |                                          |

The system distinguishes between three primary activity types, each requiring a unique approach to tracking and analysis:

1. **RECURRING:**

- **Nature:** Activities with a defined, expected schedule.
- **Significance:** Missing a scheduled day is meaningful. Streaks are a core metric.
- **Analytics:** Focus on compliance percentage and the number of missed days.

2. **OCCASIONAL:**

- **Nature:** Activities without a fixed schedule, tracked primarily by their occurrence.
- **Significance:** Measures *when* and *how often* the activity happens.
- **Analytics:** Focus on frequency, the intervals between occurrences, and overall totals.

3. **ONE\_OFF:**

- **Nature:** Single-instance events (e.g., "Doctor appointment Nov 12").
- **Significance:** Primarily for calendar visibility; no streak or frequency logic applies.

---

## 3.2 Activity Log

ActivityLog

|                |                                                        |
|----------------|--------------------------------------------------------|
| — id           |                                                        |
| — activity_id  |                                                        |
| — timestamp    |                                                        |
| — status       | DONE   SKIPPED   PARTIAL                               |
| — duration_min | optional                                               |
| — subtask_data | filled variable tree — shape defined by subtask_schema |
| — attachments  | photos, voice notes                                    |

---

## 3.3 Subtask Schema — User-Defined Variable Tree

Instead of fixed templates, every activity carries a user-built variable tree. The user defines this once when creating the activity. Each node is one of three types:

Node types

|                                                          |                                                        |
|----------------------------------------------------------|--------------------------------------------------------|
| — Group                                                  | a named container holding child nodes                  |
| — List                                                   | a repeatable container — user adds N items at log time |
| — Value                                                  | a leaf — the actual data point                         |
| value types: number   text   boolean   duration   choice |                                                        |

**Example — Gym:**

```

Gym (Group)
├── Upper Body (Group, tagged Mon/Wed)
│   └── Exercises (List)
│       ├── [each exercise] (Group)
│       │   ├── Exercise Name (Value → text)
│       │   └── Sets (List)
│       │       ├── [each set] (Group)
│       │       │   ├── Weight (Value → number, unit: kg)
│       │       │   └── Reps (Value → number)

```

### Example — Reading:

```

Reading (Group)
├── Book Title (Value → text)
├── Pages read (Value → number)
└── Takeaway (Value → text)

```

### Example — Call Mom:

Call Mom → subtask\_schema: null

The schema is opt-in. Simple activities skip it entirely.

## 4. Starter Presets

When creating an activity, the default subtask schema is a **single root node with the activity name** — nothing else. The user can then add child nodes as needed.

Optionally, the app ships pre-built variable trees for common activities that users can load as a starting point and customize:

- **Workout** — exercises list with sets, reps, weight
- **Cardio** — type, distance, duration, avg HR
- **Reading** — book, pages from-to, takeaway
- **Study** — subject, topic, duration, focus rating
- **Meditation** — technique, duration, calm rating
- **Habit (binary)** — single done/not done value
- **Counter** — single number value

These are convenience shortcuts, not constraints. The user can modify any preset — add nodes, remove nodes, rename, nest deeper — or ignore presets entirely and build from scratch.

## 5. Analytics

### Per-Activity Dashboard

Every activity — regardless of type — gets a single dashboard with these metrics:

- **Current streak + longest streak**
- **Compliance rate** (last 7d / 30d / 90d / all time) — for RECURRING this is scheduled days completed, for OCCASIONAL/ONE\_OFF this is simply total completions over time
- **Calendar heatmap** (GitHub-style) — visual history of when you did it
- **Frequency timeline** — when did this happen, how often
- **Average gap between occurrences**
- **Total count**
- **Missed-day patterns** — "you skip Tuesdays 60% of the time" (most useful for RECURRING, but available for all)
- **Goal progress bar** — if a goal is set

### Variable tree metrics

Since subtask data is user-defined, analytics are **auto-derived from the tree**. Any value node of type **number** becomes chartable automatically.

If your gym tree has "Weight" and "Reps" → the app can show volume over time, PRs, trends. If your reading tree has "Pages read" → pages/week, total pages. If your tree has no number values → only the base metrics above apply.

The user doesn't configure analytics. The app scans their logged trees, finds number values, and offers charts for them.

---

### Cross-Activity Dashboard (overall)

Zoomed-out view across everything you track.

- **Time allocation pie** — where your hours go, based on `duration_min` from logs
- **Completion momentum score** — a weekly score based on overall completion rate across all activities
- **Activity correlation** — surface patterns like "you're more consistent with Reading on days you also do Gym" (LLM-assisted, see §7)

## 6. Notifications

Seven notification types, each with a specific trigger and tone:

- **Scheduled reminder** — fires at the time the user set on the activity. Neutral, just a nudge
- **Streak-at-risk** — fires in the evening if a scheduled activity hasn't been logged yet. Slightly urgent. This is the one that saves streaks
- **Streak-broken** — fires the morning after a miss. Soft and recovery-focused, not guilt-tripping
- **Missed-day acknowledgment** — fires after a miss. Honest, non-punishing. The miss is counted in analytics silently
- **Celebration** — fires at milestones (7, 30, 100 days). Full hype
- **Insight** — fires weekly. Analytical summary generated by LLM (see §7)
- **Suggestion** — fires contextually. LLM-generated based on user's data patterns

### Planned rest days

User can mark specific days as rest without breaking their streak. For activities like gym, rest days are part of the program. The app should respect that, not punish it.

---

## 7. LLM Integration

### Hybrid approach

On-device for frequent, simple tasks. Cloud API for weekly, complex tasks.

On-device (MediaPipe LLM Inference — Gemma 2B):

- Notification copy variations

Cloud API (Claude Haiku):

- Weekly insight generation
- Activity-specific coaching
- Pattern detection in plain English

Rule-based first, LLM fallback:

- Subtask suggestions based on history

---

## Privacy

- Anonymize data before sending to cloud
  - Cloud LLM is optional in settings
  - Be explicit about what leaves the device
- 

## 8. Tech Stack

**Kotlin** — default for modern Android. Coroutines for async work, sealed classes for the variable tree node types, null safety for optional fields.

**Jetpack Compose** — declarative UI. The log entry form generates itself recursively from the variable tree — Compose handles this naturally with a composable that calls itself per node.

**MVVM + Clean Architecture** — three layers: data (Room, API calls, DataStore), domain (streak calculation, tree parsing, analytics), UI (Compose screens, ViewModels). Keeps complex logic like tree traversal and LLM data prep out of the UI layer.

**Room** — two tables. **ActivityEntity** stores activity definitions with the subtask schema as a JSON string. **ActivityLogEntity** stores every completion/skip with filled subtask data as a JSON string. Foreign key links logs to activities. Cascade delete cleans up logs when an activity is deleted.

**DataStore** — standalone settings with no relationship to activities or logs. Notification channel toggles (reminders, streak alerts, celebrations, insights, suggestions). LLM configuration (local on/off + model, cloud on/off + provider + API key + model). LLM user preferences (notification tone, free text instructions for what kind of insights the user wants). Theme preference (light/dark/system). API key stored encrypted.

**WorkManager** — background jobs that survive app restarts. Fires scheduled notifications at the right time, runs weekly LLM insight generation, computes streak data periodically.

**Vico** — Compose-native charting library. Renders calendar heatmaps, compliance rate charts, and auto-derived number charts from variable tree data. Any number value in any tree becomes chartable without chart-specific code.

**MediaPipe LLM Inference (Gemma 2B)** — on-device LLM for notification copy variations. Runs offline, no cost. User can toggle off in settings. Phase 4 addition — not MVP.

**Cloud LLM (user's choice)** — user provides their own API key and picks provider (Anthropic/OpenAI/Gemini) and model. Used for weekly insights and activity coaching. User writes free text instructions telling the LLM what kind of insights they want. Data anonymized before sending. Toggleable in settings.

**Notification channels + Direct Reply** — five channels (reminders, streak alerts, celebrations, insights, suggestions), each independently toggleable. Direct Reply shows a "Done" button on the notification for simple activities — log entry created without opening the app. Activities with subtask schemas open the app to the log form instead.

---

## 9. Features I'd Add

These came up while thinking through your spec:

1. **Quick log from notification** — tap "Done" right from the reminder, no app open needed. Reduces friction *massively* and is the #1 reason habit apps survive.
2. **Home-screen widget** — today's activities at a glance, one-tap log.
3. **Health Connect integration** — pull steps, sleep, heart rate from Google's Health Connect API. Lets the app correlate your gym data with sleep/recovery without you logging it.
4. **Calendar view** as a primary screen, not buried — humans think in calendars.
5. **Activity dependencies / chains** — "after workout, log a meal" — useful for habit stacking.
6. **Export & backup** — CSV / JSON export, encrypted backup to user's own Google Drive. Builds trust.
7. **Wear OS companion** (later) — log a workout from your watch.
8. **Themes / personalization** — small thing, big retention win.
9. **"Why?" tagging on skips** — when you skip, optionally tag a reason (tired, busy, sick, traveling). The LLM can use these to spot patterns.
10. **Rest-day intelligence** — for gym specifically, recognize that rest days *are* part of the program. Don't punish them.



